Introduction

CS 350

Dr. Joseph Eremondi

Last updated: January 6, 2025

Course Details

Course Objectives

To learn:

- Functional programming

Course Objectives

To learn:

- Functional programming
  - Recursion

Course Objectives

To learn:

- Functional programming
    - Recursion
    - Immutable data

Course Objectives

To learn:

- Functional programming
  - Recursion
  - Immutable data
  - Programming by cases

Course Objectives

To learn:

- Functional programming
    - Recursion
    - Immutable data
    - Programming by cases
    - Higher-order functions

Course Objectives

To learn:

- Functional programming
  - Recursion
  - Immutable data
  - Programming by cases
  - Higher-order functions
- How to write your own programming language

Course Objectives

To learn:

- Functional programming
    - Recursion
    - Immutable data
    - Programming by cases
    - Higher-order functions
- How to write your own programming language
    - Parsing/Abstract Syntax

Course Objectives

To learn:

- Functional programming
  - Recursion
  - Immutable data
  - Programming by cases
  - Higher-order functions
- How to write your own programming language
  - Parsing/Abstract Syntax
  - Desugaring

Course Objectives

To learn:

- Functional programming
  - Recursion
  - Immutable data
  - Programming by cases
  - Higher-order functions
- How to write your own programming language
  - Parsing/Abstract Syntax
  - Desugaring
  - Evaluation

Course Objectives

To learn:

- Functional programming
  - Recursion
  - Immutable data
  - Programming by cases
  - Higher-order functions
- How to write your own programming language
  - Parsing/Abstract Syntax
  - Desugaring
  - Evaluation

To change how you *think* about programming

Course Notes / Textbook

- Course Notes: *Functional Languages, Interpreters and Types*

Course Notes / Textbook

- Course Notes: *Functional Languages, Interpreters and Types*
  - Work in progress, so let me know if you discover any errors/issues

Course Notes / Textbook

- Course Notes: *Functional Languages, Interpreters and Types*
    - Work in progress, so let me know if you discover any errors/issues
    - Covers the first half of the course (Functional Programming)

Course Notes / Textbook

- Course Notes: *Functional Languages, Interpreters and Types*
    - Work in progress, so let me know if you discover any errors/issues
    - Covers the first half of the course (Functional Programming)
- Textbook: *Programming Languages: Application and Interpretation*, 3rd edition, by Shriram Krishnamurthi

Course Notes / Textbook

- Course Notes: *Functional Languages, Interpreters and Types*
    - Work in progress, so let me know if you discover any errors/issues
    - Covers the first half of the course (Functional Programming)
- Textbook: *Programming Languages: Application and Interpretation*, 3rd edition, by Shriram Krishnamurthi
    - aka PLAI

Course Notes / Textbook

- Course Notes: *Functional Languages, Interpreters and Types*
    - Work in progress, so let me know if you discover any errors/issues
    - Covers the first half of the course (Functional Programming)
- Textbook: *Programming Languages: Application and Interpretation*, 3rd edition, by Shriram Krishnamurthi
    - aka PLAI
    - Freely available online, pdf in UR Courses

Course Notes / Textbook

- Course Notes: *Functional Languages, Interpreters and Types*
  - Work in progress, so let me know if you discover any errors/issues
  - Covers the first half of the course (Functional Programming)
- Textbook: *Programming Languages: Application and Interpretation*, 3rd edition, by Shriram Krishnamurthi
  - aka PLAI
  - Freely available online, pdf in UR Courses
  - Covers the second half of the course (Implementing Programming Languages)

Course Notes / Textbook

- Course Notes: *Functional Languages, Interpreters and Types*
  - Work in progress, so let me know if you discover any errors/issues
  - Covers the first half of the course (Functional Programming)
- Textbook: *Programming Languages: Application and Interpretation*, 3rd edition, by Shriram Krishnamurthi
  - aka PLAI
  - Freely available online, pdf in UR Courses
  - Covers the second half of the course (Implementing Programming Languages)
    - We'll follow loosely

Course Communication

- Everything on URCourses

Course Communication

- Everything on URCourses
  - Announcements

Course Communication

- Everything on URCourses
  - Announcements
  - Assignments and Handin

Course Communication

- Everything on URCourses
  - Announcements
  - Assignments and Handin
  - Textbook, Slides, Videos

Course Communication

- Everything on URCourses
  - Announcements
  - Assignments and Handin
  - Textbook, Slides, Videos
  - Discussion Forum

Course Communication

- Everything on URCourses
  - Announcements
  - Assignments and Handin
  - Textbook, Slides, Videos
  - Discussion Forum
  - Messages to Instructor

Course Communication

- Everything on URCourses
  - Announcements
  - Assignments and Handin
  - Textbook, Slides, Videos
  - Discussion Forum
  - Messages to Instructor
- Do NOT ask programming/conceptual questions by email/private message

Course Communication

- Everything on URCourses
  - Announcements
  - Assignments and Handin
  - Textbook, Slides, Videos
  - Discussion Forum
  - Messages to Instructor
- Do NOT ask programming/conceptual questions by email/private message
  - Use the discussion forum

Course Communication

- Everything on URCourses
    - Announcements
    - Assignments and Handin
    - Textbook, Slides, Videos
    - Discussion Forum
    - Messages to Instructor
- Do NOT ask programming/conceptual questions by email/private message
    - Use the discussion forum
    - If you're wondering, others are too

Course Communication

- Everything on URCourses
  - Announcements
  - Assignments and Handin
  - Textbook, Slides, Videos
  - Discussion Forum
  - Messages to Instructor
- Do NOT ask programming/conceptual questions by email/private message
  - Use the discussion forum
  - If you're wondering, others are too
  - EXCEPTION: when you can't ask your question without revealing your solution to the assignment

Office Hours

- Location: RIC 317

Office Hours

- Location: RIC 317
  - Take the elevator, go across the bridge and through the door

Office Hours

- Location: RIC 317
  - Take the elevator, go across the bridge and through the door
  - Or take the stairs by the “Church Turing Thesis” wall and vending machines

Office Hours

- Location: RIC 317
    - Take the elevator, go across the bridge and through the door
    - Or take the stairs by the “Church Turing Thesis” wall and vending machines
- Tues at 10:00

Office Hours

- Location: RIC 317
  - Take the elevator, go across the bridge and through the door
  - Or take the stairs by the “Church Turing Thesis” wall and vending machines
- Tues at 10:00
- Thurs at 11:30

Office Hours

- Location: RIC 317
  - Take the elevator, go across the bridge and through the door
  - Or take the stairs by the “Church Turing Thesis” wall and vending machines
- Tues at 10:00
- Thurs at 11:30
- Check pinned announcement on URCourses

Grading Scheme

- 35% assignments

Grading Scheme

- 35% assignments
- 15% midterm

Grading Scheme

- 35% assignments
- 15% midterm
  - In-class

Grading Scheme

- 35% assignments
- 15% midterm
  - In-class
  - Wednesday, Feb 12

Grading Scheme

- 35% assignments
- 15% midterm
  - In-class
  - Wednesday, Feb 12
- 40% final

Grading Scheme

- 35% assignments
- 15% midterm
    - In-class
    - Wednesday, Feb 12
- 40% final
- 10% Participation

Participation

- You get marks just for showing up

Participation

- You get marks just for showing up
- Each lecture is 0.5%, to a maximum of 10%

Participation

- You get marks just for showing up
- Each lecture is 0.5%, to a maximum of 10%
- Each forum post is 0.5%, maximum of 5%

Participation

- You get marks just for showing up
- Each lecture is 0.5%, to a maximum of 10%
- Each forum post is 0.5%, maximum of 5%
    - e.g. You need to attend half the lectures for full marks

Participation

- You get marks just for showing up
- Each lecture is 0.5%, to a maximum of 10%
- Each forum post is 0.5%, maximum of 5%
  - e.g. You need to attend half the lectures for full marks
- Assignment 0 is 1% (counts as 2 lectures)

Assignment 0

- Due Monday, On URCourses

Assignment 0

- Due Monday, On URCourses
- Get your Dr. Racket Set up

Assignment 0

- Due Monday, On URCourses
- Get your Dr. Racket Set up
  - Download from `https://download.racket-lang.org/`

Assignment 0

- Due Monday, On URCourses
- Get your Dr. Racket Set up
  - Download from `https://download.racket-lang.org/`
  - Install the plt file

Assignment 0

- Due Monday, On URCourses
- Get your Dr. Racket Set up
    - Download from `https://download.racket-lang.org/`
    - Install the plt file
    - Full instructions on URCourses

Assignment 0

- Due Monday, On URCourses
- Get your Dr. Racket Set up
  - Download from `https://download.racket-lang.org/`
  - Install the plt file
  - Full instructions on URCourses
- Get your handin account setup

Assignment 0

- Due Monday, On URCourses
- Get your Dr. Racket Set up
  - Download from `https://download.racket-lang.org/`
  - Install the plt file
  - Full instructions on URCourses
- Get your handin account setup
  - Username and password under “Feedback” for “Login Details”

Assignment 0

- Due Monday, On URCourses
- Get your Dr. Racket Set up
  - Download from `https://download.racket-lang.org/`
  - Install the plt file
  - Full instructions on URCourses
- Get your handin account setup
  - Username and password under “Feedback” for “Login Details”
  - Submit the assignment using

Assignments 1-6

- Six bi-weekly assignments

Assignments 1-6

- Six bi-weekly assignments
- Due Fridays at 5pm (17:00)

Assignments 1-6

- Six bi-weekly assignments
- Due Fridays at 5pm (17:00)
    - No extensions

Assignments 1-6

- Six bi-weekly assignments
- Due Fridays at 5pm (`17:00`)
  - No extensions
  - Lowest two grades dropped

Assignments 1-6

- Six bi-weekly assignments
- Due Fridays at 5pm (17:00)
    - No extensions
    - Lowest two grades dropped
        - Last two are “optional” if you’re happy with your marks

Assignments (ctd.)

- Mostly programming

Assignments (ctd.)

- Mostly programming
  - Some conceptual questions

Assignments (ctd.)

- Mostly programming
  - Some conceptual questions
- Score based on running tests

Assignments (ctd.)

- Mostly programming
  - Some conceptual questions
- Score based on running tests
  - Some public (included in assignment)

Assignments (ctd.)

- Mostly programming
  - Some conceptual questions
- Score based on running tests
  - Some public (included in assignment)
  - Some private (only known by me)

Assignments (ctd.)

- Mostly programming
  - Some conceptual questions
- Score based on running tests
  - Some public (included in assignment)
  - Some private (only known by me)
  - Code doesn't run $\rightarrow$ no marks

Assignments (ctd.)

- Mostly programming
  - Some conceptual questions
- Score based on running tests
  - Some public (included in assignment)
  - Some private (only known by me)
  - Code doesn't run → no marks
- Some points for style/documentation/etc.

Assignments (ctd.)

- Mostly programming
  - Some conceptual questions
- Score based on running tests
  - Some public (included in assignment)
  - Some private (only known by me)
  - Code doesn’t run → no marks
- Some points for style/documentation/etc.
  - Sample based marking

Submitting Assignments via Handin

- Unlimited submissions before deadline

Submitting Assignments via Handin

- Unlimited submissions before deadline
- After a submission, you get:

Submitting Assignments via Handin

- Unlimited submissions before deadline
- After a submission, you get:
  - A score

Submitting Assignments via Handin

- Unlimited submissions before deadline
- After a submission, you get:
    - A score
    - A list of which public tests failed, along with expected results

Submitting Assignments via Handin

- Unlimited submissions before deadline
- After a submission, you get:
    - A score
    - A list of which public tests failed, along with expected results
- **READ THE TEST RESULTS** to know what you need to fix

Syntax and Type Errors

- **ALL CODE CONTAINING SYNTAX OR TYPE ERRORS WILL GET A GRADE OF ZERO**

Syntax and Type Errors

- **ALL CODE CONTAINING SYNTAX OR TYPE ERRORS WILL GET A GRADE OF ZERO**
    - The handin tool will give an error message if your file doesn't compile

Syntax and Type Errors

- **ALL CODE CONTAINING SYNTAX OR TYPE ERRORS WILL GET A GRADE OF ZERO**
  - The handin tool will give an error message if your file doesn’t compile
- If your code has a type or syntax error, you can try submitting again

Syntax and Type Errors

- **ALL CODE CONTAINING SYNTAX OR TYPE ERRORS WILL GET A GRADE OF ZERO**
    - The handin tool will give an error message if your file doesn't compile
- If your code has a type or syntax error, you can try submitting again
- It's better to submit code that fails some tests than to submit code with type or syntax errors

Resources for This Course

- There are few resources available for you to learn about `flit`

Resources for This Course

- There are few resources available for you to learn about `flit`
  - THIS IS BY DESIGN

Resources for This Course

- There are few resources available for you to learn about `flit`
  - THIS IS BY DESIGN
- You will learn to think about code

Resources for This Course

- There are few resources available for you to learn about `flit`
  - THIS IS BY DESIGN
- You will learn to think about code
  - With your brain

Resources for This Course

- There are few resources available for you to learn about `flit`
    - THIS IS BY DESIGN
- You will learn to think about code
    - With your brain
    - Not with copy/paste or AI

Resources for This Course

- There are few resources available for you to learn about `flit`
  - THIS IS BY DESIGN
- You will learn to think about code
  - With your brain
  - Not with copy/paste or AI
- `flit` is a small language

Resources for This Course

- There are few resources available for you to learn about `flit`
  - THIS IS BY DESIGN
- You will learn to think about code
  - With your brain
  - Not with copy/paste or AI
- `flit` is a small language
  - All the features/functions you need will be described in the course notes

Resources for This Course

- There are few resources available for you to learn about `flit`
  - THIS IS BY DESIGN
- You will learn to think about code
  - With your brain
  - Not with copy/paste or AI
- `flit` is a small language
  - All the features/functions you need will be described in the course notes
- There are many languages built using Racket

Resources for This Course

- There are few resources available for you to learn about `flit`
  - THIS IS BY DESIGN
- You will learn to think about code
  - With your brain
  - Not with copy/paste or AI
- `flit` is a small language
  - All the features/functions you need will be described in the course notes
- There are many languages built using Racket
  - Copying code will probably get you a `0`

Academic Integrity

- You are to complete all assignments **individually**

Academic Integrity

- You are to complete all assignments **individually**
- Some (easy) questions on assignments are individualized to you, and must be completed for your assignment to be graded.

Academic Integrity

- You are to complete all assignments **individually**
- Some (easy) questions on assignments are individualized to you, and must be completed for your assignment to be graded.
- All of the following are considered cheating:

Academic Integrity

- You are to complete all assignments **individually**
- Some (easy) questions on assignments are individualized to you, and must be completed for your assignment to be graded.
- All of the following are considered cheating:
    - Using code from a local or online AI tool

Academic Integrity

- You are to complete all assignments **individually**
- Some (easy) questions on assignments are individualized to you, and must be completed for your assignment to be graded.
- All of the following are considered cheating:
  - Using code from a local or online AI tool
  - Copying code from a website like StackOverflow

Academic Integrity

- You are to complete all assignments **individually**
- Some (easy) questions on assignments are individualized to you, and must be completed for your assignment to be graded.
- All of the following are considered cheating:
    - Using code from a local or online AI tool
    - Copying code from a website like StackOverflow
    - Viewing another student’s solution

Academic Integrity

- You are to complete all assignments **individually**
- Some (easy) questions on assignments are individualized to you, and must be completed for your assignment to be graded.
- All of the following are considered cheating:
  - Using code from a local or online AI tool
  - Copying code from a website like StackOverflow
  - Viewing another student's solution
- ChatGPT DOES NOT UNDERSTAND FLIT

Academic Integrity

- You are to complete all assignments **individually**
- Some (easy) questions on assignments are individualized to you, and must be completed for your assignment to be graded.
- All of the following are considered cheating:
  - Using code from a local or online AI tool
  - Copying code from a website like StackOverflow
  - Viewing another student's solution
- ChatGPT DOES NOT UNDERSTAND FLIT
  - Getting AI code to work is harder than just doing the assignment

Academic Integrity

- You are to complete all assignments **individually**
- Some (easy) questions on assignments are individualized to you, and must be completed for your assignment to be graded.
- All of the following are considered cheating:
  - Using code from a local or online AI tool
  - Copying code from a website like StackOverflow
  - Viewing another student's solution
- ChatGPT DOES NOT UNDERSTAND FLIT
  - Getting AI code to work is harder than just doing the assignment

Don't set yourself up for failure on the exams

Course Content

Course Objectives

Programming language genealogy and design.
Imperative, functional, and object-oriented language paradigms.
Context-free grammars and syntax trees.
Data types, control structures, exception handling, data abstraction, information hiding, and non-determinism.
Program representation, translation, and execution.
Functional programming: advantages, constructs, closures, and higher-order operations.
Parallel programming.

Course Objectives

Programming language genealogy and design.
Imperative, **functional**, and object-oriented language paradigms.
Context-free grammars and syntax trees.
Data types, control structures, exception handling, data abstraction, information hiding, and non-determinism.
Program representation, translation, and execution.
Functional programming: advantages, constructs, closures, and higher-order operations.
Parallel programming.

- *First half of the class: Programming in Flit*

Course Objectives

Programming language genealogy and design.
Imperative, functional, and **object-oriented language paradigms.**
Context-free grammars and syntax trees.
Data types, control structures, exception handling, data abstraction, information hiding, and non-determinism.
Program representation, translation, and execution.
Functional programming: advantages, constructs, closures, and higher-order operations.
Parallel programming.

- *Second half of class: implementing interpreters*

Common Questions

- Am I going to use Racket in Industry?

Common Questions

- Am I going to use Racket in Industry?
    - No

Common Questions

- Then why are we learning it

Common Questions

- Then why are we learning it
    - Don’t know what you’ll be using in industry

Common Questions

- Then why are we learning it
    - Don’t know what you’ll be using in industry
    - Learn a small language that shows core concept

Common Questions

- Then why are we learning it
  - Don't know what you'll be using in industry
  - Learn a small language that shows core concept
  - Learn the pieces that languages are made of

Common Questions

- Then why are we learning it
  - Don’t know what you’ll be using in industry
  - Learn a small language that shows core concept
  - Learn the pieces that languages are made of
    - How to see past syntax

Common Questions

- Then why are we learning it
    - Don't know what you'll be using in industry
    - Learn a small language that shows core concept
    - Learn the pieces that languages are made of
        - How to see past syntax
        - Understand what programs *mean*

Common Questions

- Then why are we learning it
  - Don’t know what you’ll be using in industry
  - Learn a small language that shows core concept
  - Learn the pieces that languages are made of
    - How to see past syntax
    - Understand what programs *mean*
  - Then you’ll be able to learn whatever language you need in industry

Common Questions

- Do you really expect me to learn a new language in just one term?

Common Questions

- Do you really expect me to learn a new language in just one term?
    - Yes. **This is a programming languages class**.

Common Questions

- Do you really expect me to learn a new language in just one term?
    - Yes. **This is a programming languages class**.
    - The language we're learning is small (really just functions and pattern matching)

When might this course be useful?

- You might get hired to

When might this course be useful?

- You might get hired to
    - Write a JavaScript Node server using callbacks to handle requests

When might this course be useful?

- You might get hired to
  - Write a JavaScript Node server using callbacks to handle requests
  - Maintain a python library full of lambdas and list comprehension

When might this course be useful?

- You might get hired to
  - Write a JavaScript Node server using callbacks to handle requests
  - Maintain a python library full of lambdas and list comprehension
  - Add to Discord’s chat system built in Elixr

When might this course be useful?

- You might get hired to
  - Write a JavaScript Node server using callbacks to handle requests
  - Maintain a python library full of lambdas and list comprehension
  - Add to Discord's chat system built in Elixr
  - Write an Android GUI using Kotlin, Jetpack Compose and function composition

When might this course be useful?

- You might get hired to
  - Write a JavaScript Node server using callbacks to handle requests
  - Maintain a python library full of lambdas and list comprehension
  - Add to Discord's chat system built in Elixr
  - Write an Android GUI using Kotlin, Jetpack Compose and function composition
  - Write Rust at Amazon, or write an iPhone app in Swift, and need to use `Option` instead of null pointers

When might this course be useful?

- You might get hired to
  - Write a JavaScript Node server using callbacks to handle requests
  - Maintain a python library full of lambdas and list comprehension
  - Add to Discord's chat system built in Elixr
  - Write an Android GUI using Kotlin, Jetpack Compose and function composition
  - Write Rust at Amazon, or write an iPhone app in Swift, and need to use `Option` instead of null pointers
- I can't teach you all of Kotlin, JavaScript, Elixr, Rust, Swift, Clojure, Scala, etc. in one term

When might this course be useful?

- You might get hired to
  - Write a JavaScript Node server using callbacks to handle requests
  - Maintain a python library full of lambdas and list comprehension
  - Add to Discord's chat system built in Elixr
  - Write an Android GUI using Kotlin, Jetpack Compose and function composition
  - Write Rust at Amazon, or write an iPhone app in Swift, and need to use `Option` instead of null pointers
- I can't teach you all of Kotlin, JavaScript, Elixr, Rust, Swift, Clojure, Scala, etc. in one term
  - But I can teach you what they're made of

The Science of Computer Science

- How to think about programs in a rigorous way

The Science of Computer Science

- How to think about programs in a rigorous way
  - Programs as mathematical objects

The Science of Computer Science

- How to think about programs in a rigorous way
  - Programs as mathematical objects
  - Equations with programs

The Science of Computer Science

- How to think about programs in a rigorous way
  - Programs as mathematical objects
  - Equations with programs
- Problem solving with types

The Science of Computer Science

- How to think about programs in a rigorous way
  - Programs as mathematical objects
  - Equations with programs
- Problem solving with types
  - How to make sure you handle all the cases

The Science of Computer Science

- How to think about programs in a rigorous way
  - Programs as mathematical objects
  - Equations with programs
- Problem solving with types
  - How to make sure you handle all the cases
    - Making invalid states unrepresentable

The Science of Computer Science

- How to think about programs in a rigorous way
  - Programs as mathematical objects
  - Equations with programs
- Problem solving with types
  - How to make sure you handle all the cases
    - Making invalid states unrepresentable
  - How the shape of a problem can guide to its solution

The Science of Computer Science

- How to think about programs in a rigorous way
  - Programs as mathematical objects
  - Equations with programs
- Problem solving with types
  - How to make sure you handle all the cases
    - Making invalid states unrepresentable
  - How the shape of a problem can guide to its solution
  - Using types to communicate properties of code

The Science of Computer Science

- How to think about programs in a rigorous way
    - Programs as mathematical objects
    - Equations with programs
- Problem solving with types
    - How to make sure you handle all the cases
        - Making invalid states unrepresentable
    - How the shape of a problem can guide to its solution
    - Using types to communicate properties of code
- This is radically different from the kind of coding you have done so far